

WEEKLY PROGRESS REPORT – WEEK 01

Project: NovaBank – Banking Information System

Internship: Core Java Intern

Organization: upSkill Campus in collaboration with UniConverge Technologies Pvt. Ltd. (The IoT Academy)

1. Overview

During the first phase of the internship project, the primary focus was on understanding the NovaBank Banking Information System requirements, organizing the Java project structure, implementing the core model and service components, and establishing persistent local data storage. The work was aimed at building a functional console-based banking application using Core Java.

2. Activities Completed

- Reviewed the Banking Information System requirements and identified the major entities and operations required by the application.
- Organized the project using Java packages such as model, service, storage, util, and exception.
- Implemented the Customer, Account, Transaction, and Admin model classes.
- Implemented ID generation for customers and account numbers through the IdGenerator utility.
- Implemented DataStorage using Java object serialization for persistent storage of customer, account, and transaction records.
- Developed the BankService layer containing customer registration, login, profile management, balance checking, deposits, withdrawals, transfers, transaction history, and password management.
- Added validation and exception handling for invalid amounts, insufficient balance, incorrect credentials, and invalid input.
- Tested the application through the console and resolved compilation and runtime issues during development.

3. Milestones Achieved

- Established the basic modular architecture of the NovaBank application.
- Implemented the core banking entities and business logic.
- Enabled persistent storage using Java Serialization.
- Completed the primary customer banking operations.
- Reached a functional console-based prototype suitable for further testing and documentation.

4. Challenges and Hurdles

- Managing package structure and dependencies between model, service, storage, and utility classes.
- Resolving compilation and method-signature issues while integrating multiple Java classes.
- Handling input validation and preventing invalid banking operations such as withdrawals beyond the available balance.

- Maintaining application data between runs using serialized files.

5. Solutions / Approach

The issues were addressed by separating responsibilities across packages, checking class and method signatures systematically, adding validation before modifying account balances, and centralizing data persistence through the DataStorage utility. The application was repeatedly compiled and tested after changes to identify and resolve integration issues.

6. Lessons Learned

- Improved understanding of object-oriented design and separation of responsibilities in Java.
- Learned how Java Serialization can be used for simple persistent storage.
- Improved debugging by tracing compilation errors and runtime behavior across multiple classes.
- Learned the importance of validating input before performing financial operations.
- Developed practical experience in structuring a multi-class Java application.

7. Next Week's Planned Work

- Perform broader functional testing of all banking operations.
- Refine error handling and user interaction where required.
- Verify data persistence across application restarts.
- Prepare project documentation, screenshots, and UML diagrams.
- Maintain the project repository using Git and GitHub.